דף סיכום בחינה

מזהה סטודנט: [redacted]

מזהה קורס: 20594   שם קורס: מערכות הפעלה

מספר שאלה	ניקוד מירבי	ציון
1	26.00	24.00
2	15.00	15.00
3	14.00	14.00
4	5.00	5.00
5	5.00	5.00
6	5.00	5.00
7	5.00	5.00
8	5.00	5.00
9	5.00	5.00
10	5.00	5.00
11	5.00	0.00
12	5.00	5.00

ציון בחינה סופי : 93.00

הבחינה הבדוקה בעמודים הבאים

[redacted]
מספר התלמיד הנבחן
רשום את כל תשע הספרות

ש [redacted]

ת.ז.: [redacted] מספר סידורי: 12

האוניברסיטה הפתוחה

כ"ה באב תשפ"ב

22

מס' שאלון - 525

באוגוסט 2022

סמסטר 2022ב

מס' מועד 95

20594 / 4

שאלון בחינת גמר

20594 - מערכות הפעלה

משך בחינה: 3 שעות

בשאלון זה **13** עמודים

מבנה הבחינה:

קראו בעיון לפני שתתחילו בפתרון הבחינה!

א. המבחן מורכב משלושה חלקים.

ב. בחלקים א ו - ב מופיעות שאלות פתוחות. ענו תשובות מלאות, בכתב קריא ובקיצור נמרץ. אין חובה להשתמש בכל השורות המוקצות לצורך התשובות, אך אין לחרוג מהמקום המוקצה.

ג. בחלק ג (שאלות אמריקאיות) עליכם לבחור בכל פעם בתשובה יחידה מבין התשובות המוצעות ולהקיף בעיגול את אות התשובה שבחרתם.

את התשובות לכל השאלות יש לכתוב בשאלון הבחינה.

חומר עזר:
כל חומר עזר אסור בשימוש, פרט למחשבון, שאינו אוצר מידע.

בהצלחה !!!

החזירו
למשגיח את השאלון
וכל עזר אחר שקיבלתם בתוך מחברת התשובות

שאלון 525

95.53.2 M1

הנחיות למבחן

בחינה בעקרונות מערכות הפעלה

קראו בעיון לפני שתתחילו בפתרון הבחינה!

1. בחלקים א׳ ו ב׳ מופיעות שאלות פתוחות. ענו **תשובות מלאות**, **בכתב קריא** ו**בקיצור נמרץ**. אין חובה להשתמש בכל השורות המוקצות לצורך התשובות, אך **אין לחרוג מהמקום המוקצה.**
2. בחלק ג׳ (שאלות אמריקאיות) עליכם לבחור בכל פעם בתשובה **יחידה הנכונה ביותר** מבין התשובות.
3. כל חומר עזר אסור לשימוש בזמן הבחינה פרט למחשבון פשוט
4. משך הבחינה - שלוש שעות.

בהצלחה!

חלק א (55 נקודות)

ענו על השאלות 1-3.

שאלה 1 (26 נקודות)

במערכת עם שני תהליכים p0 ו-p1 , משתמשת בפרוטוקול הבא למניעה הדדית:

```
// all variables visible from p0 and p1
entrance_intents[0] = false
entrance_intents[1] = false
turn = 0
```

P0	p1
`P0:`	`p1:`
`while (true){`	`while (true){`
`  entrance_intents[0] = true;`	`  entrance_intents[1] = true;`
`  while (entrance_intents[1]){`	`  while (entrance_intents[0]){`
`    if (turn == 1){`	`    if (turn == 0){`
`      entrance_intents[0]=false;`	`      entrance_intents[1]=false;`
`      while (turn == 1){`	`      while (turn == 0){`
`        // busy wait`	`        // busy wait`
`      }`	`      }`
`      entrance_intents[0]=true;`	`      entrance_intents[1]=true;`
`    }`	`    }`
`  }`	`  }`
`  // critical section`	`  // critical section`
`  ...`	`  ...`
`  turn = 1;`	`  turn = 0;`
`  entrance_intents[0] = false;`	`  entrance_intents[1] = false;`
`}`	`}`

א. (8 נק׳)

האם הפרוטוקול מבטיח מניע הדדית? הוכיחו שכן או הפריכו ע״י דוגמא נגדית.

✕

תשובה והסבר:

כן, תהליך מחוץ לקטע הקריטי לא ימנע מהשני
להיכנס (שכן מסמן שהוא לא מעוניין ואז השני יכול להיכנס)
אך שני תהליכים לא יכולים להיות בתוך הקטע הקריטי אלא שניהם היו
מסומנים כלא מעוניינים - מסמנים זה לזה שכן לאחר
מה תמיד מסמן שהוא מעוניין לפני שהוא נכנס לקטע הקריטי.
ואין הרעבה - לפי סעיף ג.

ההסבר אינו מספיק, משתנה אחד אינו מספיק

לא לכתוב בשוליים

שאלון 525

95.53.2 M1

נא לא לכתוב בשוליים

ב. (7 נק׳)

האם יתכן קיפאון (במובן של העדר התקדמו- אף תהליך לא יוכל להיכנס לקטע קריטי) במערכת?
הוכיחו שלא או תנו דוגמא שכן.

תשובה והסבר:

לכן, לא יכול להיות ששני התהליכים ישארו ב while של שורה 7 בו זמנית (שכן המשתנה turn הוא 1 או 0.
וכאשר אחד התהליכים נמצא ב while של שורה 7 הוא מסמן לשני שהוא לא מעוניין ולכן התהליך השני יכול להתקדם.

ג. (5 נק׳)

האם תיתכן הרעבה (במובן של המתנה לא מוגבלת בזמן בניסיון כניסה לקטע קריטי) במערכת?

תשובה והסבר:

לא, ברגע שאחד התהליכים מסיים את הקטע הקריטי הוא לא יוכל להיכנס אליו שוב כל עוד התהליך השני מעוניין גם הוא להיכנס (שכן המשתנה turn יהיה של התהליך השני).

ד. (6 נק׳)

האם תשובתך לסעיף ב׳ תשתנה אם נמחק את שורות 5 ו 11(שאר שורות הקוד שהיו בתוך ה IF לא נמחקות) עבור שני התהליכים ?

תשובה והסבר:

לא, בכל מקרה לא יהיה קיפאון שכן עדיין לא יכול להיות מצב שבו תהליך ממתין לביצוע של תהליך אחר.
(במידה ושני התהליכים מעוניינים להיכנס, זה שהתור לא שלו יישאר ב while ולכן ימנע [illegible] מהשני להמשיך להתקדם.

נא לא לכתוב בשוליים

נא לא לכתוב בשוליים

שאלה 2 (15 נקודות)

א. 7 נק'

הסבירו מהי הבעייתיות של האלגוריתם LRU כאשר מממשים אותו באמצעות רשימה מקושרת של דפים להחלפה.

תשובה והסבר:

הבעייתיות היא העלות הגבוהה שנובעת מהצורך להעתיק דפים ממויינים לפי זמן התייחסות, שכן בכל פנייה לדף יש לעדכן את המיקום שלו ברשימה.

ב. 8 נק'

האם באלגוריתם LRU ייתכן שיהיו יותר שגיאות דף כאשר מקצים לתהליך יותר מסגרות זיכרון פיזיות?

הוכיחו שלא או תנו דוגמא שכן(רשימת התייחסויות לדפי זיכרון).

תשובה והסבר:

לא יתכן, שכן כאשר לתהליך יותר מסגרות זיכרון LRU יעתיק, בנוסף לדפים שהיה מעתיק אם היה לו פחות מסגרות זיכרון, עוד דפים שזמן התייחסות שלהם רחוק יותר.

כלומר כל אותם דפים שהיו מועתקים אם היה לו פחות מסגרות זיכרון, יועתקו גם כאשר יהיה לו יותר ולכן לא יתכן שיהיה יותר שגיאות דף.

15
(2)

נא לא לכתוב בשוליים

שאלון 525

95.53.2 M1

נא לא לכתוב בשוליים

שאלה 3 (14 נקודות)

נתונה מערכת קבצים דמוית ext2 מבוססת I –NODES עם 12 מצביעים ישירים והשאר single indirect, double indirect, triple indirect (מצביע אחד בכל סוג).

גודל הבלוק הוא 1 KB , אורך הכתובת 4 Byte.

(5 נק') א. שרטטו והסבירו מבנה של קובץ ספרייה של מערכת קבצים הנ"ל .

שרטוט :

.	5
..	4
file1	8
⋮	
filen	15

הסבר :

בכל ספרייה יש את שמות הקבצים שקיימים בה ומספר ה inode שלהם, כאשר '.' מסמן את התיקייה הנוכחית, '..' את התיקייה הקודמת. (הדגמתי מספרי inode לדוגמה)

(6 נק') ב. מהו מספר הבלוקים המקסימאלי שצריך לקרוא מתוך מערכת הקבצים הנ"ל (אחרי פתיחת הקובץ) על מנת לבצע חיפוש בקובץ שגודלו 780 KB?

תשובה והסבר :

בכל בלוק יכל להכיל כתובות של $\frac{1024}{4}=256$ בלוקים אחרים. בשביל הקובץ הנתון צריך 780 בלוקים רק נצטרך גם לקרוא את ה single indirect, ה double indirect ועוד שני בלוקים פנימיים שלה מגדיר ה double indirect סה"כ 780+1+1+2=784 בלוקים.

(המשך השאלה בעמוד הבא)

נא לא לכתוב בשוליים

שאלון 525 95.53.2 M1

נא לא לכתוב בשוליים

(3 נק') ג. זמן ממוצע של קריאת בלוק אחד מן הדיסק הוא 10 ms (כולל כל מרכיבי זמן הגישה). בכמה מילישניות קריאת בלוק מספר 5 של הקובץ מסעיף קודם תהיה מהירה יותר מקריאת בלוק מספר 275 בעקבות הרצת הקוד הבא?

```
int fd = open("text", O_RDONLY);
lseek(fd, 5*BLOCK_SIZE, SEEK_SET);
read(fd, char arr[], BLOCK_SIZE);
lseek(fd,  275*BLOCK_SIZE, SEEK_SET);
read(fd, char arr[], BLOCK_SIZE);
```

החישוב והסבר:

לבלוק החמישי יש מצביע ישיר לכן נקרא אותו ב 10ms.

בשביל הבלוק מספר 275 נצטרך קודם לקרוא את ה double indirect ובלוק פנימי שהוא מצביע עליו - סה"כ נקרא 3 בלוקים לכן ייקח 30ms

וקיבלנו שההפרש הוא 20ms.

המשך הבחינה בעמוד הבא

נא לא לכתוב בשוליים

שאלון 525 95.53.2 M1

נא לא לכתוב בשוליים

חלק ב (25 נקודות)

ענו על השאלות 4-8.

(5 נקודות לשאלה; 25 נקודות לחלק זה).

שאלה 4

חברה מספקת שירותי מחשוב ענן CLOUD COMPUTING ללקוחות שונים ע"י יצירת קונטיינר נפרד לכל לקוח. מערכת ההפעלה הבסיסית בענן שעליה נוצרים קונטיינרים היא LINUX . בתוך הקונטיינר כל לקוח צריך להתקין את מה שהוא רוצה. האם לקוחות יוכלו להתקין בתוך הקונטיינרים שלהם מערכת הפעלה(לא וירטואלית) ששונה ממערכת ההפעלה שבה נוצרים הקונטיינרים?

תתיחסו ל-2 המקרים:

א. מערכת ההפעלה עם הגרעין(KERNEL) משלה ששונה מן הגרעין של המערכת הבסיסית שרצה בענן.

ב. מערכת ההפעלה עם אותו הגרעין(KERNEL) כמו הגרעין של המערכת הבסיסית שרצה בענן.

תשובה והסבר:

הקונטיינרים ומה שרץ בהם בסופו של דבר חייבים להשתמש בשירותים של מערכת ההפעלה הבסיסית שהם רצים עליה.

לכן לא נוכל להתקין בתוך הקונטיינרים מערכת הפעלה עם גרעין שונה משל המערכת הבסיסית (למשל windows.linux).

אבל אם למערכת ההפעלה שנרצה להתקין אותו גרעין כמו של המערכת הבסיסית אז נוכל.

5
(4)

נא לא לכתוב בשוליים

95.53.2 M1 שאלון 525

נא לא לכתוב בשוליים

שאלה 5

במערכת XV6 קיימת קריאת מערכת KILL(PID) שמאפשרת להרוג את התהליך ע״פ PID שלו.

מה יהיה הפלט אחרי הרצת הפקודות הבאות במערכת XV6 שתומכת בקונטיינרים?

```
unshare(PID_NS);
int pid = fork();
if(pid == 0){
  pid = fork();
  if(pid == 0) {
    sleep(99999); // very long sleep
    printf(stdout, "GrandSon PID=%d \n", getpid());
    }
   else {
     sleep(99999); // very long sleep
     printf(stdout, "Son PID=%d \n", getpid());
    }
  }
 else {
   sleep(9999); // sleep enough for creating all son processes, but they are still sleeping
   kill(pid+1);
 }
```

ניתן להניח שכל קריאות מערכת מצליחות ושלא נוצרו תהליכים נוספים במערכת מרגע הרצת הקוד(חוץ מאלה שהקוד יוצר) ושמערכת מקצה PID עוקב כל פעם שנוצר תהליך חדש.

תשובה והסבר:

התהליך הבן שנוצר ב PID_NS החדש יקבל את ה PID שלו שיהיה
7, לאחר מכן הוא יצור את הנכד, אולם שהאבא
עושה kill(pid+1) הוא הורג את הנכד שכן זה ה PID
של הנכד ב PID_NS של האבא.
לכן סה״כ נקבל שרק הבן ידפיס: son PID = 7

5
(5)

נא לא לכתוב בשוליים

95.53.2 M1 שאלון 525

נא לא לכתוב בשוליים

שאלה 6

מה יודפס בעקבות הרצת קטע קוד הבא בתוך מערכת XV6 שתומכת בקונטיינרים, ניתן להניח שכל קריאות מערכת מצליחות:

```
unshare(MOUNT_NS);
int pid = fork();
if (pid == 0){
mkdir("dirS");
mount("internal_fs_a","dirS",0); //mount a preformatted internal_fs_a
int fd = open("dirS/file.txt",O_CREATE|O_RDWR);
write(fd ,"123456789\n",10);
close (fd);
}
if (pid > 0)sleep(10000); // wait for child to finish
sleep(10000); // both processes will sleep
if(pid == 0){
   if (open("dirS/file.txt",O_RDONLY) < 0) printf(stderr, "son failed to open\n");
   else printf(stderr, "son succeeded\n");
}
If(pid>0){
  if (open("dirS/file.txt",O_RDONLY) < 0) printf(stderr, "father failed to open\n");
  else printf(stderr, "father succeeded\n");
}
```

תשובה והסבר:

ה unshare נעשה לפני ה fork לכן גם האב
וגם הבן יהיו באותו mount_ns ולכן שניהם
יוכלו לפתוח את הקובץ ויצליחו:
father succeeded ; son succeedes

5
(6)

נא לא לכתוב בשוליים

שאלון 525 95.53.2 M1

נא לא לכתוב בשוליים

שאלה 7

נתונה מערכת הפעלה עם ~~5~~ 4 סוגי משאבים שונים, עותק אחד מכל סוג ומאפשרת הרצה במקביל של 3 תהליכים לכל היותר. במערכת קיים פרוטוקול הבא שימוש במשאבים:

מה מבין התנאים הבאים מבטיח את מניעת deadlock במערכת עם 4 סוגי RESOURCES שונים:

1. כל process יוכל לבקש לא יותר מ 2 משאבים(כמובן מסוגים שונים, אחד מכל סוג) בסה״כ. RESOURCES
2. אחרי ש processes קיבל **את כל** RESOURCES שביקש, הוא ישחרר את כולם בתוך 10 שניות לכל היותר.
3. כל process ישחרר את כל המשאבים(RESOURCES) שהוא החזיק בהם(כולל אמצעי סנכרון) לפני שיסתיים.

5
(7)

האם במערכת הזאת ייתכן קיפאון DEADLOCK ? הוכיחו שלא או תנו דוגמא איך יכול להיווצר קיפאון.

תשובה והסבר:

מתכן קיפאון שכן אין הגבלה על מס' המשאבים של תהליך יכול להחזיק, לכן אם תהליך אחד יחזיק את משאבים 1,2,3 ~~[illegible]~~ ומבקש את משאב 4. ותהליך שני מחזיק את משאב 4 ומבקש את משאב 3, קיבלנו קיפאון.

נסתכל על מצב שבו תהליך 1 מחזיק את משאב 1 וצריך את משאב 2 ותהליך 2 מחזיק את משאב 2 וצריך את משאב 1. קיבלנו קיפאון למרות שתנאי 1 מתקיים וגם תנאי 2 מתקיים (התהליכים עדיין לא קיבלו מה שביקשו) ותנאי 3 מתקיים (התהליכים לא יכולים להסתיים שכן הם בקיפאון). לכן תנאי לא טוב.

* חלק השאלה שאומרת שלושת הסעיפים מתייחס למצב לפני מחיקת אחד הסעיפים, אם לבדוק את האפשרות לא מונעת קיפאון, לא לפתור את סעיף ולמנוע.

נא לא לכתוב בשוליים

שאלון 525 95.53.2 M1

נא לא לכתוב בשוליים

שאלה 8

במערכת לינוקס הורצו 2 התכניות הבאות:

//Program 1	//Program 2

#define AR_SIZE 2048
main() {
    int i , j ,a[AR_SIZE][AR_SIZE];
    for (i=0; i < AR_SIZE; i++)
        for (j=0; j < AR_SIZE; j++)
            a[i][j]=0;
}
	#define AR_SIZE 2048
	main() {
	    int i , j , a[AR_SIZE][AR_SIZE];
	    for (i=0; i < AR_SIZE; i++)
	        for (j=0; j < AR_SIZE; j++)
	            a[j][i]=0;
	}

שימו לב, ההבדל היחיד בין התכניות הוא בשורה אחת בלבד. שתי התכניות הורצו במקביל(בו-זמנית) מספר רב של פעמים ובכל פעם נמדד הזמן מתחילת ההרצה עד לסיומה. התברר כי בכל פעם התכנית הראשונה מסתיימת בזמן קצר יותר מאשר התכנית השנייה. ידוע שגודל המערך הדו-ממדי **a** נכנס כולו לקבוצת העבודה של התהליך. מה הסיבה **__הסבירה ביותר__** לכך?

הערה: ניתן לומר שהתכניות נמצאות בתנאים זהים מבחינת הקצאת משאבים על-ידי מערכת הפעלה.

5
(8)

תשובה והסבר:

בהרצה של התכנית הראשונה יש ניצול טוב
יותר של זכרון המטמון, זכרון המטמון לא
מספיק גדול בשביל המערך אך בתכנית השנייה יש
פחות ניצול של מה שקיים במטמון.

המשך הבחינה בעמוד הבא

נא לא לכתוב בשוליים

נא לא לכתוב בשוליים

חלק ג (20 נקודות)

ענו על השאלות 9-12.

(5 נקודות לשאלה; 20 נקודות לחלק זה).

שאלה 9

כדי לבחור את הדף המתאים לפינוי מהזיכרון הראשי הוצע האלגוריתם הבא :לכל מסגרת (frame) בזיכרון מוצמד מונה .כאשר דף חדש מגיע למסגרת,ערכו של מונה זה נקבע להיות 0 ובכל פניה לדף המונה מועלה באחד. כאשר יש צורך לפנות דף מהזיכרון ,בוחרים תמיד את הדף שהמונה המוצמד למסגרת שלו מכיל את הערך הנמוך ביותר .האם אלגוריתם זה הוא טוב לניהול דפדוף בזיכרון?

א. כן, המונה משקף את מידת נחיצותו של הדף. ככל שערכו נמוך ,כך מספר הפניות לדף היה נמוך יותר ולכן הוא מתאים לפינוי.

ב. כן, מונה גבוהה עם הפרש גדול מצביע על דף הנמצא בשימוש מתמיד שבטוח לא כדאי לפנות אותו.

ג. לא, אלגוריתם זה אינו מאפשר כלל הישארות של דפים חדשים כי המונה שלהם תמיד 0 ולכן יפונו מיד.

ד. לא .מצב המונה עלול לשקף מציאות היסטורית שאינה קיימת עוד.

5
(9)

שאלה 10

התנאי לכך שיהיה ניתן לתזמן n משימות מחזוריות במערכת זמן אמת (real-time) הוא :

$$\sum_i \frac{C_i}{P_i} \leq 1$$

כאשר Pi הוא זמן המחזור (period) ו Ci הוא זמן ה CPU של המשימה ה-i -ית .

על איזו הנחה סמויה הנוסחה מתבססת?

יש לבחור תשובה אחת :

5
(10)

א. זמן החלפת המשימות (context switch) הוא זניח.

ב. לכל i בין 1 ל n -מתקיים Pi= Ci .

ג. לא ניתן לתזמן אף משימה כל עוד לא מתקיים שוויון בנוסחה הנ"ל.

ד. לכל i בין 1 ל n -מתקיים Pi < Ci.

המשך הבחינה בעמוד הבא

נא לא לכתוב בשוליים

נא לא לכתוב בשוליים

שאלה 11

בחרו טענה נכונה לגבי מערכות מבוססת ה-microkernel:

א. מערכות ה-microkernel רק מספקת לרמת אפליקציה את משאבי המערכת כאוסף מצומצם מאוד של אבסטרקציות. כתוצאה מכך לאפליקציות ניתן מרחב תמרון גדול מאוד בכל הנוגע לניהול משאבים.

ב. גרעין מערכת ההפעלה המבוססת microkernel מתפקד כדוור המריץ בקשות של לקוחות (תהליכים) אל שרתים עצמאיים המפוזרים במערכת, כגון שרתי מערכת הקבצים, מנהל הזיכרון וכו'. האבסטרקציות ממומשות ע"י שרתים בלבד.

ג. מערכות ההפעלה המבוססות microkernel מתאפיינות באספקה מספר קטן של שירותים של מערכת כולה בעקבות היכולות המצומצמות של microkernel .

ד. כל התשובות הקודמות הן נכונות.

0
(11)

שאלה 12

מערכת ההפעלה מסוימת נותנת לתהליכים בעלי עדיפות גבוהה פרוסת זמן(quantum , time slice) ארוכים, ואילו לתהליכים בעלי עדיפות נמוכה מקצה quantum קצר. במערכת הזאות תהליכים בעלי עדיפות גבוהה הם בדרך כלל התהליכים אינטרקטיביים ואילו תהליכים בעלי עדיפות נמוכה הם התהליכים המבצעים חישובים (CPU bounded) . כתוצאה מכך, במקרים שיש במערכת יש הרבה תהליכים חישוביים ,יקרו הרבה החלפות תוכן (context switches) . כמובן שהדבר לא רצוי ,מכיוון שהיינו רוצים לתת לתהליכים חישוביים לרוץ כמה שיותר זמן וללא החלפות תוכן תדירות.

בחרו שיטת תזמון הפותרת בעיה זו:

א. shortest remaining time next

ב. shortest process next

ג. fair share scheduling

ד. first come first served

5
(12)

בהצלחה!

נא לא לכתוב בשוליים

שאלון 525 95.53.2 M1

נא לא לכתוב בשוליים